

Git Bash im Selbststudium

Durchgängige Übung: Eine Café-Website versionieren

Ziel	Die drei Hauptbereiche Working Directory, Staging Area und Repository praktisch nachvollziehen und passende Git-Befehle sicher anwenden.
Projekt	Eine Café-Website wird schrittweise aufgebaut: zuerst index.html, dann styles.css, danach script.js. Jede sinnvolle Änderung wird versioniert.

Voraussetzungen
Git ist installiert und Git Bash kann geöffnet werden.
Ein einfacher Texteditor steht zur Verfügung, z. B. VS Code, Notepad++ oder Editor.

Merksatz

Working Directory = hier bearbeitest du Dateien. **Staging Area** = hier sammelst du gezielt Änderungen für den nächsten Commit. **Repository** = hier liegen die gespeicherten Versionen (Commits).

Git-Befehle, die in dieser Übung vorkommen

Befehl	Bedeutung	Typische Frage
git init	Erstellt ein neues Repository im aktuellen Ordner.	Ist dieser Ordner schon ein Git-Projekt?
git status	Zeigt den Zustand von Working Directory und Staging Area.	Was ist neu, geändert oder schon staged?
git add <Datei>	Legt Änderungen in die Staging Area.	Welche Datei soll in den nächsten Commit?
git commit -m "..."	Speichert den gestagten Stand dauerhaft im Repository.	Welche Version möchte ich festhalten?

git log --oneline	Zeigt die Commit-Historie kompakt an.	Welche Versionen gibt es bereits?
git diff	Zeigt noch nicht gestagte Änderungen.	Was habe ich im Working Directory verändert?
git diff --staged	Zeigt gestagte Änderungen.	Was genau liegt schon in der Staging Area?

1. Projektordner erstellen und Repository initialisieren

Arbeite in Git Bash. Erstelle einen neuen Ordner und wechsle hinein.

Selbstkontrolle
Nach _____ wurde ein leeres Repository erstellt.
_____ zeigt typischerweise an: 'On branch master' oder 'On branch main' sowie 'nothing to commit'.
Es gibt noch keine versionierten Dateien.

2. Erste Datei anlegen: index.html

Erstelle im Projektordner eine Datei mit dem Namen index.html und füge folgenden Inhalt ein:

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Café Sonnenschein</title>
</head>
<body>
  <h1>Willkommen im Café Sonnenschein</h1>
  <p>Auf dieser Website entsteht Schritt für Schritt eine kleine Café-Präsentation.</p>
</body>
</html>
```

Speichere die Datei. Anschließend prüfst du mit Git, was passiert ist. Welcher Git-Befehl ist dafür geeignet?

Beobachtung
index.html liegt jetzt im Working Directory.
Die Datei ist noch nicht in der Staging Area.
_____ zeigt die Datei als 'untracked file'.
_____ zeigt bei untracked Dateien normalerweise noch nichts an.

3. Die HTML-Datei stagen und committen

Jetzt soll die erste Version der Website dauerhaft im Repository gespeichert werden.

1. _____
2. _____
3. _____
4. _____
5. _____
6. _____

Selbstkontrolle
Nach git _____ index.html liegt die Datei in der Staging Area.
_____ zeigt genau den Inhalt, der gleich committed wird.
Nach dem _____ ist die Arbeitsumgebung wieder sauber: 'nothing to commit, working tree clean'.
_____ --oneline zeigt genau einen Commit.

4. HTML-Datei ändern und den Unterschied prüfen

Erweitere den Inhalt der index.html. Ersetze den body-Bereich durch folgende Version:

```
<body>
  <h1>Willkommen im Café Sonnenschein</h1>
  <p>Auf dieser Website entsteht Schritt für Schritt eine kleine Café-Präsentation.</p>
  <h2>Unsere Highlights</h2>
  <ul>
    <li>Frischer Kaffee</li>
    <li>Hausgemachte Kuchen</li>
    <li>Gemütliche Atmosphäre</li>
  </ul>
</body>
```

Prüfe danach den Zustand.

1. _____
2. _____

Beobachtung

Die Datei index.html ist jetzt geändert, aber noch nicht gestaged.

_____ zeigt die Änderung zwischen Working Directory und Staging Area.

Du siehst hier also genau, was du seit dem letzten Staging verändert hast.

5. Geänderte HTML-Version committen

1. _____
2. _____
3. `git diff --staged`
4. _____ -m "Startseite des Cafés um Inhalte erweitert"
5. _____ --online

Damit existieren nun bereits zwei gespeicherte Versionen der Startseite des Cafés im Repository.

6. CSS-Datei ergänzen und mit HTML verknüpfen

Erstelle nun eine neue Datei styles.css mit folgendem Inhalt:

```
body {  
  font-family: Arial, sans-serif;  
  background-color: #fff8f0;  
  margin: 30px;  
}  
  
h1 {  
  color: #7a3e1d;  
}  
  
p {  
  color: #4a3b32;  
}  
  
ul {  
  background-color: white;  
  padding: 15px 30px;  
  border-radius: 8px;  
  border: 1px solid #d8c3a5;
```

```
}
```

Ergänze zusätzlich in der index.html im head-Bereich diese Zeile:

```
<link rel="stylesheet" href="styles.css">
```

Prüfe nun wieder den Zustand des Projekts.

1. _____
2. _____
3. _____ (zwei Dateien gleichzeitig der Staging Area hinzufügen)
4. _____
5. _____
6. _____ -m "Café-Layout gestaltet und eingebunden"
7. _____ --online

Wichtig

Es wurden diesmal zwei Dateien gemeinsam committed.

index.html war geändert, styles.css war neu.

Beide landen erst nach git add in der Staging Area.

Erst git commit speichert den gestagten Zustand im Repository.

7. JavaScript-Datei ergänzen und mit HTML verknüpfen

Erstelle jetzt eine Datei script.js mit folgendem Inhalt:

```
function begruessungAnzeigen() {  
  const ausgabe = document.getElementById("ausgabe");  
  ausgabe.textContent = "Heute gibt es frischen Apfelstrudel und Cappuccino."  
}
```

Erweitere die index.html im body um einen Button und eine Ausgabestelle:

```
<button onclick="begruessungAnzeigen()">Tagesangebot anzeigen</button>  
<p id="ausgabe"></p>
```

Binde außerdem die Datei script.js vor dem schließenden body-Tag ein:

```
<script src="script.js"></script>
```

Danach wieder Git verwenden:

1. _____
2. _____
3. _____ (zwei Dateien gleichzeitig der Staging Area hinzufügen)
4. _____
5. _____ -m "Tagesangebot per JavaScript ergänzt"
6. _____ --online

Selbstkontrolle

Die Datei script.js ist neu, index.html wurde erneut geändert.

Nach dem Commit sollte git log --online jetzt vier Commits anzeigen.

Die Café-Website besteht nun aus HTML, CSS und JavaScript.

8. Drei Hauptbereiche bewusst nachvollziehen

Ordne die folgenden Situationen den drei Bereichen zu.

Situation	Bereich
Du hast styles.css gespeichert, aber noch kein git add ausgeführt.	
Du hast index.html mit git add index.html vorgemerkt.	
Du hast git commit -m "..." ausgeführt.	
git diff zeigt Änderungen an.	
git diff --staged zeigt Änderungen an.	

9. Abschlussauftrag

Bearbeite nun selbstständig die Café-Website weiter:

- Ändere die Überschrift in index.html, z. B. auf „Café Sonnenschein in Graz“.
- Ergänze in styles.css eine Regel für den Button, damit er besser zum Café-Design passt.
- Passe den Text an, der in script.js ausgegeben wird, z. B. auf ein anderes Tagesangebot.

- Prüfe nach jeder Änderung mit `git status` und `git diff` den Zustand.
- Führe am Ende einen weiteren sinnvollen Commit aus.

Reflexionsfragen

Wann befindet sich eine Datei nur im Working Directory?

Warum ist die Staging Area nützlich und nicht überflüssig?

Worin besteht der Unterschied zwischen `git diff` und `git diff --staged`?

Warum ist es sinnvoll, mehrere kleine, verständliche Commits zu erstellen?

Kurzfazit

In dieser Übung wurde ein Git-Projekt von Grund auf aufgebaut. Die Lernenden haben erlebt, wie Dateien einer kleinen Café-Website zunächst im Working Directory entstehen oder verändert werden, anschließend gezielt in die Staging Area übernommen und schließlich per Commit dauerhaft im Repository gespeichert werden. Durch `index.html`, `styles.css` und `script.js` wurde der Ablauf mehrfach in einem zusammenhängenden Beispiel wiederholt.